

Deep Learning (CAI-466)

Objectives:

-
- **Progress in Deep Reinforcement Learning**
 - **The Environment**
 - **Markov Decision Processes (MDPs)**
-

Progress in deep reinforcement learning

The use of artificial neural networks for RL problems started around the 1990s. One of the classics is the backgammon-playing computer program, *TD-Gammon*, created by Gerald Tesauro et al. *TD-Gammon* learned to play backgammon by learning to evaluate table positions on its own through RL. Even though the techniques implemented aren't precisely considered DRL, *TD-Gammon* was one of the first widely reported success stories using ANNs to solve complex RL problems.

In 2004, Andrew Ng et al. developed an autonomous helicopter that taught itself to fly stunts by observing hours of human-experts flights. They used a technique known as *inverse reinforcement learning*, in which an agent learns from expert demonstrations. The same year, Nate Kohl and Peter Stone used a class of DRL methods known as *policy-gradient methods* to develop a soccer-playing robot for the RoboCup tournament. They used RL to teach the agent forward motion. After only three hours of training, the robot achieved the fastest forward-moving speed of any robot with the same hardware.

There were other successes in the 2000s, but the field of DRL really only started growing after the DL field took off around 2010. In 2013 and 2015, Mnih et al. published a couple of papers presenting the DQN algorithm. DQN learned to play Atari games from raw pixels. Using a convolutional neural network (CNN) and a single set of hyperparameters, DQN performed better than a professional human player in 22 out of 49 games.

This accomplishment started a revolution in the DRL community: In 2014, Silver et al. released the deterministic policy gradient (DPG) algorithm, and a year later Lillicrap et al. improved it with deep deterministic policy gradient (DDPG). In 2016, Schulman et al. released trust region policy optimization (TRPO) and generalized advantage estimation (GAE) methods, Sergey

Levine et al. published Guided Policy Search (GPS), and Silver et al. demoed AlphaGo. The following year, Silver et al. demonstrated AlphaZero. Many other algorithms were released during these years: double deep Q-networks (DDQN), prioritized experience replay (PER), proximal policy optimization (PPO), actor-critic with experience replay (ACER), asynchronous advantage actor-critic (A3C), advantage actor-critic (A2C), actor-critic using Kronecker-factored trust region (ACKTR), Rainbow, Unicorn (these are actual names, BTW), and so on. In 2019, Oriol Vinyals et al. showed the AlphaStar agent beat professional players at the game of StarCraft II. And a few months later, Jakub Pachocki et al. saw their team of Dota-2-playing bots, called Five, become the first AI to beat the world champions in an e-sports game.

Thanks to the progress in DRL, we've gone in two decades from solving backgammon, with its 10^{20} perfect-information states, to solving the game of Go, with its 10^{170} perfect-information states, or better yet, to solving StarCraft II, with its 10^{270} imperfect-information states. It's hard to conceive a better time to enter the field.

The Environment

The most real world examples can be expressed as RL environment. A common way to represent decision making process in RL is by modeling the problem using a mathematical framework known as Markov decision process (MDPs).

Examples of MDPs environment

Here are a few more examples of environments that are powered by underlying MDPs

Description	Observation space	Sample observation	Action space	Sample action	Reward function
Hotter, colder: Guess a randomly selected number using hints.	Int range 0–3. 0 means no guess yet submitted, 1 means guess is lower than the target, 2 means guess is equal to the target, and 3 means guess is higher than the target.	2	Float from –2000.0–2000.0. The float number the agent is guessing.	–909.37	The reward is the squared percentage of the way the agent has guessed toward the target.

Lunar lander: Navigate a lander to its landing pad.	An eight-element vector with ranges: from $[-\text{Inf}, -\text{Inf}, -\text{Inf}, -\text{Inf}, -\text{Inf}, -\text{Inf}, 0, 0]$ to $[\text{Inf}, \text{Inf}, \text{Inf}, \text{Inf}, \text{Inf}, \text{Inf}, 1, 1]$. First element is the x position, the second the y position, the third is the x velocity, the fourth is the y velocity, fifth is the vehicle's angle, sixth is the angular velocity, and the last two values are Booleans indicating legs contact with the ground.	$[0.36, 0.23, -0.63, -0.10, -0.97, -1.73, 1.0, 0.0]$	Int range 0–3. No-op (do nothing), fire left engine, fire ² main engine, fire right engine.		Reward for landing is 200. There's a reward for moving from the top to the landing pad, for crashing or coming to rest, for each leg touching the ground, and for firing the engines.
Pong: Bounce the ball past the opponent, and avoid letting the ball pass you.	A tensor of shape 210, 160, 3. Values ranging 0–255. Represents a game screen image.	$[[[246, 217, 64], [55, 184, 230], [46, 231, 179], \dots, [28, 104, 249], [25, 5, 22], [173, 186, 1]], \dots]]$	Int range 0–5. Action 0 is No-op, 1 is Fire, 2 is up, 3 is right, 4 is Left, 5 is Down. Notice how some actions ³ don't affect the game in any way. In reality the paddle can only move up or down, or not move.		The reward is a 1 when the ball goes beyond the opponent, and a –1 when your agent's paddle misses the ball.

Humanoid: Make robot run as fast as possible and not fall.	A 44-element (or more, depending on the implementation) vector. Values ranging from $-\text{Inf}$ to Inf . Represents the positions and velocities of the robot's joints.	[0.6, 0.08, 0.9, 0.0, 0.0, 0.0, 0.0, 0.0, 0.045, 0.0, 0.47, ..., 0.32, 0.0, -0.22, ..., 0.]	A 17-element vector. Values ranging from $-\text{Inf}$ to Inf . Represents the forces to apply to the robot's joints.	[-0.9, -0.06, 0.6, 0.6, 0.6, -0.06, -0.4, -0.9, 0.5, -0.2, 0.7, -0.9, 0.4, -0.8, -0.1, 0.8, -0.03]	The reward is calculated based on forward motion with a small penalty to encourage a natural gait.
Cart pole: Balance a pole in a cart.	A four-element vector with ranges: from [-4.8, $-\text{Inf}$, -4.2, $-\text{Inf}$] to [4.8, Inf , 4.2, Inf]. First element is the cart position, second is the cart velocity, third is pole angle in radians, fourth is the pole velocity at tip.	[-0.16, -1.61, 0.17, 2.44]	Int range 0-1. 0 means push cart left, 1 means push cart right.	0	The reward is 1 for every step taken, including the termination step.

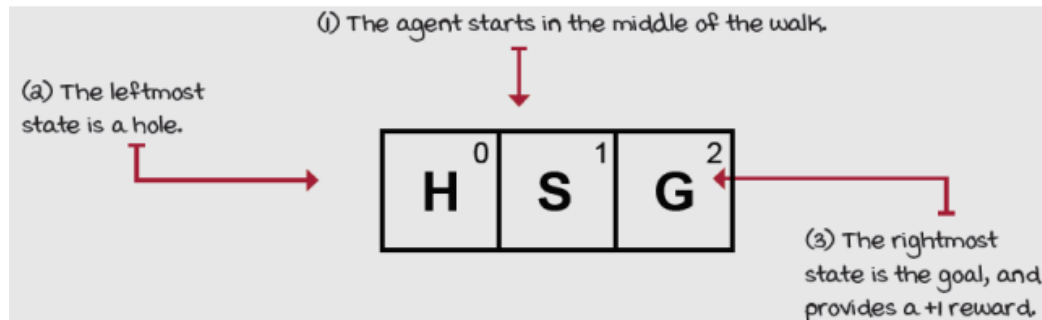
The bandit walk environment

Let's make these concepts concrete with our first RL environment call it the bandit walk (BW).

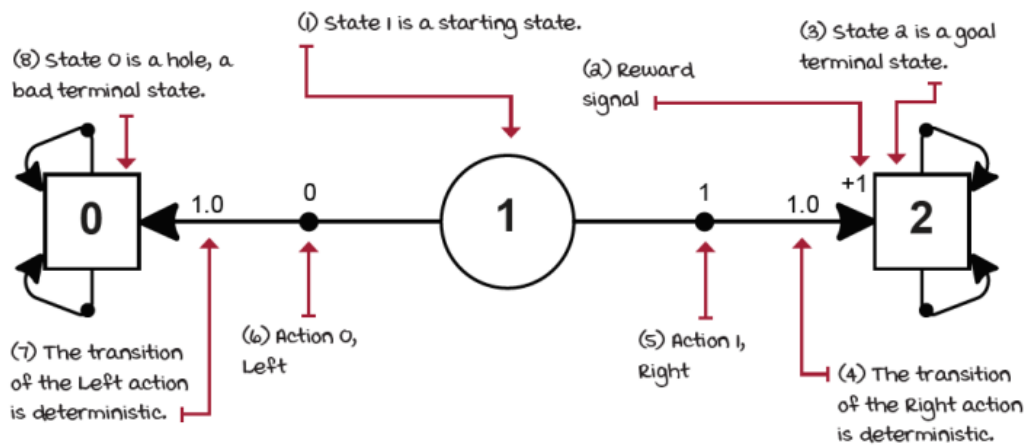
BW is a simple grid-world (GW) environment. GWs are a common type of environment for studying RL algorithms that are grids of any size. GWs can have any model (transition and reward functions) you can think of and can make any kind of actions available.

But, they all commonly make move actions available to the agent: Left, Down, Right, Up (or West, South, East, North, which is more precise because the agent has no heading and usually has no visibility of the full grid, but cardinal directions can also be more confusing). And, of course, each action corresponds with its logical transition: Left goes left, and Right goes right. Also, they all tend to have a fully observable discrete state and observation spaces (that is, state equals observation) with integers representing the cell id location of the agent. A "walk" is a

special case of grid-world environments with a single row. The bandit walk (BW) is a walk with three states, but only one non-terminal state. Environments that have a single non-terminal state are called “**bandit**” environments. The BW environment has just two actions available: a right. The reward signal is a +1 when landing on the rightmost cell, 0 otherwise. The agent starts in the middle cell.



Bandit walk graph



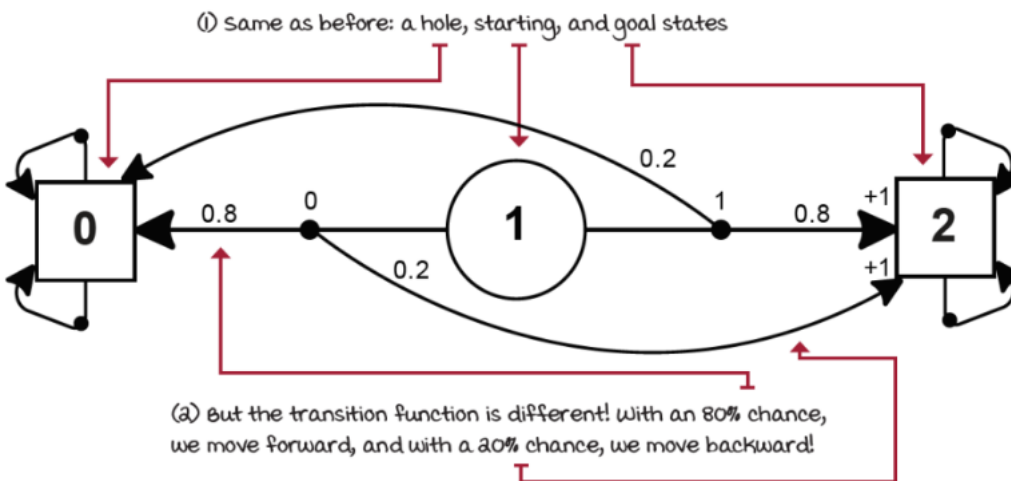
We can also represent this environment in a table form.

State	Action	Next state	Transition probability	Reward signal
0 (Hole)	0 (Left)	0 (Hole)	1.0	0
0 (Hole)	1 (Right)	0 (Hole)	1.0	0
1 (Start)	0 (Left)	0 (Hole)	1.0	0
1 (Start)	1 (Right)	2 (Goal)	1.0	+1
2 (Goal)	0 (Left)	2 (Goal)	1.0	0
2 (Goal)	1 (Right)	2 (Goal)	1.0	0

Bandit slippery walk environment

Let's say the surface of the walk is slippery and each action has a 20% chance of sending the agent backwards and call this environment the **bandit slippery walk (BSW)**. BSW is still a one-row-grid world, a walk, a corridor, with only Left and Right actions available. Again, three states and two actions. The reward is the same as before, however, the transition function is different: 80% of the time the agent moves to the intended cell, and 20% of time in the opposite direction. A depiction of this environment would look as follows.

Bandit slippery walk graph

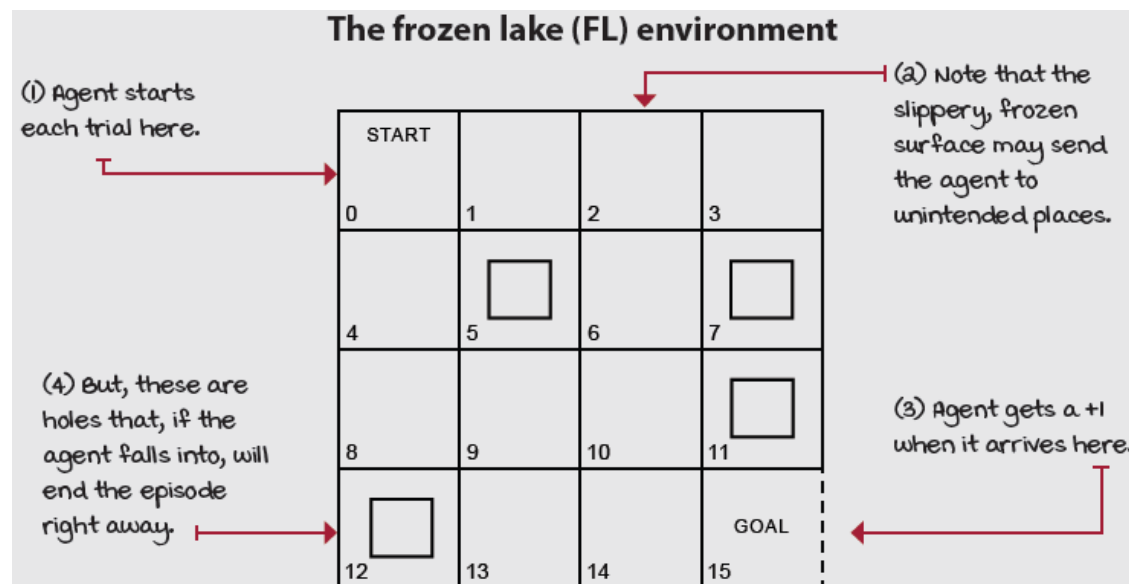


How do we know that the action effects are stochastic? How do we represent the “slippery” part of this problem? The *table* representations can help us with that.

State	Action	Next state	Transition probability	Reward signal
0 (Hole)	0 (Left)	0 (Hole)	1.0	0
0 (Hole)	1 (Right)	0 (Hole)	1.0	0
1 (Start)	0 (Left)	0 (Hole)	0.8	0
1 (Start)	0 (Left)	2 (Goal)	0.2	+1
1 (Start)	1 (Right)	2 (Goal)	0.8	+1
1 (Start)	1 (Right)	0 (Hole)	0.2	0
2 (Goal)	0 (Left)	2 (Goal)	1.0	0
2 (Goal)	1 (Right)	2 (Goal)	1.0	0

The frozen Lake Environment

This is another, more challenging problem for which we will build an MDP in this chapter. This environment is called the frozen lake (FL). FL is a simple grid-world (GW) environment. It also has discrete state and action spaces. However, this time, four actions are available: move Left, Down, Right, or Up. The task in the FL environment is to go from a start location to a goal location while avoiding falling into holes. The challenge is similar to the BSW, in that the surface of the FL environment is slippery, it's a frozen lake after all. But the environment itself is larger. The FL is a 4×4 grid (it has 16 cells, ids 0–15). The agent shows up in the START cell every new episode. Reaching the GOAL cell gives a +1 reward; anything else is a 0. Because the surface is slippery, the agent moves only a third of the time as intended. ***The other two-thirds are split evenly in orthogonal directions.*** For example, if the agent chooses to move down, there's a 33.3% chance it moves down, 33.3% chance it moves left, and 33.3% chance it moves right. There's a fence around the lake, so if the agent tries to move out of the grid world, it will bounce back to the cell from which it tried to move. There are four holes in the lake. If the agent falls into one of these holes, it's game over.

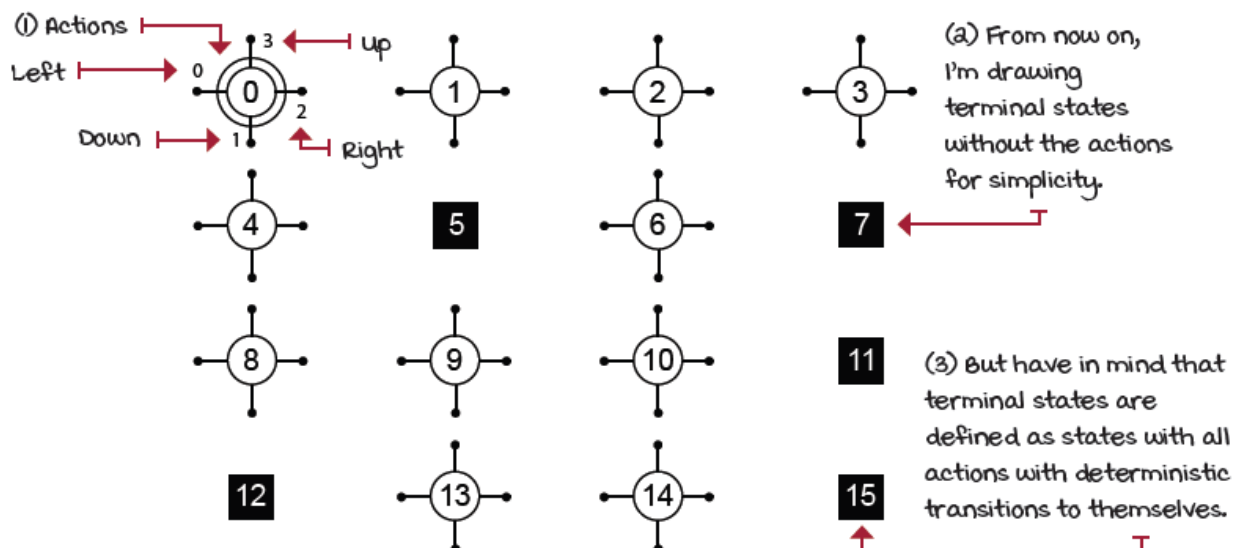


For the BW, BSW, and FL environments, the state is composed of a single variable containing the id of the cell where the agent is at any given time. The agent's location cell id is a discrete variable. But state variables can be of any kind, and the set of variables can be larger than one.

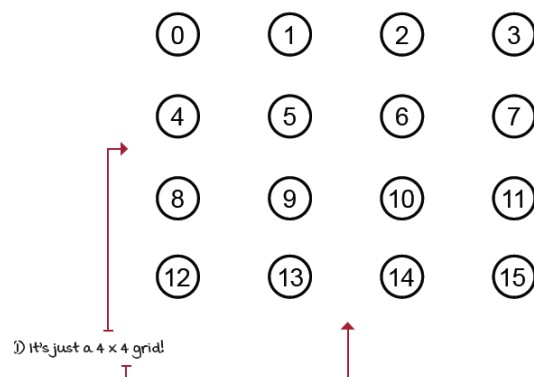
With this state representation for the BW, BSW, and FL environments we have 3, 3, or 16 cells, the agent can be at any given time, then we have 3, 3, and 16 possible states in the state space. We can set the ids of each cell starting from zero, going left to right, top to bottom.

Actions:

MDPs make available a set of actions A that depends on the state. That is, there might be actions that aren't allowed in a state—in fact, A is a function that takes a state as an argument; that is, $A(s)$. This function returns the set of available actions for state s . In the BW, BSW, and FL environments, actions are singletons representing the direction the agent will attempt to move. In FL, there are four available actions in all states: Up, Down, Right, or Left. There's one variable per action, and the size of the action space is four.



States in the FL contain a single variable indicating the id of the cell in which the agent is at any given time step



The Markov Property

The probability of the next state, given the current state and action, is independent of the history of interactions. This memoryless property of MDPs is known as the **Markov property**: the probability of moving from one state s to another state s on two separate occasions, given the same action a , is the same regardless of all previous states or actions encountered before that point. for grid-world environments, only the state id location of the agent is sufficient.

The diagram shows the equation $P(S_{t+1} | S_t, A_t) = P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots)$ with four annotations and arrows:

- (1) The probability of the next state ... (points to $P(S_{t+1})$)
- (2) ... given the current state and current action ... (points to S_t, A_t)
- (3) ... will be the same ... (points to the right side of the equation)
- (4) ... as if you give it the entire history of interactions. (points to the history terms $S_{t-1}, A_{t-1}, \dots$)

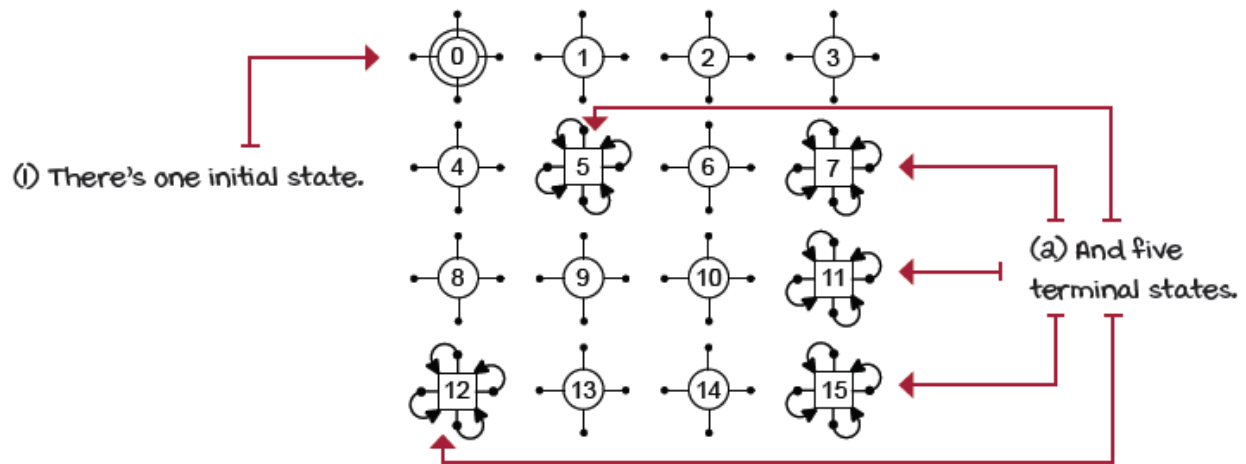
The set of all states in the MDP is denoted S^+ . There is a subset of S^+ called the set of *starting* or *initial states*, denoted S_i . To begin interacting with an MDP, we draw a state from S_i from a probability distribution. This distribution can be anything, but it must be fixed throughout training: that is, the probabilities must be the same from the first to the last episode of training and for agent evaluation.

There's a unique state called the **absorbing** or **terminal state**, and the set of all non-terminal states is denoted S . Now, while it's common practice to create a single terminal state (a sink state) to which all terminal transitions go, this isn't always implemented this way.

A terminal state is a special state: it must have all available actions transitioning, with probability 1, to itself, and these transitions must provide no reward.

In the FL environment, for instance, there's only one starting state (which is state 0) and five terminal states (or five states that transition to a single terminal state, whichever you prefer). For clarity, I use the convention of multiple terminal states (5, 7, 11, 12, and 15) for the illustrations and code; again, each terminal state is a separate terminal state.

States in the frozen lake environment



Transition function: Consequences of agent actions

The way the environment changes as a response to actions is referred to as the *state-transition probabilities*, or more simply, the *transition function*, and is denoted by $T(s, a, s')$. The transition function T maps a transition tuple s, a, s' to a probability; that is, you pass in a state s , an action a , and a next state s' , and it'll return the corresponding probability of transition from state s to state s' when taking action a .

① The transition function is defined...

(a) ... as the probability of transitioning to state s' at time step t ...

(b) ... given action a was selected on state s in the previous time step $t-1$.

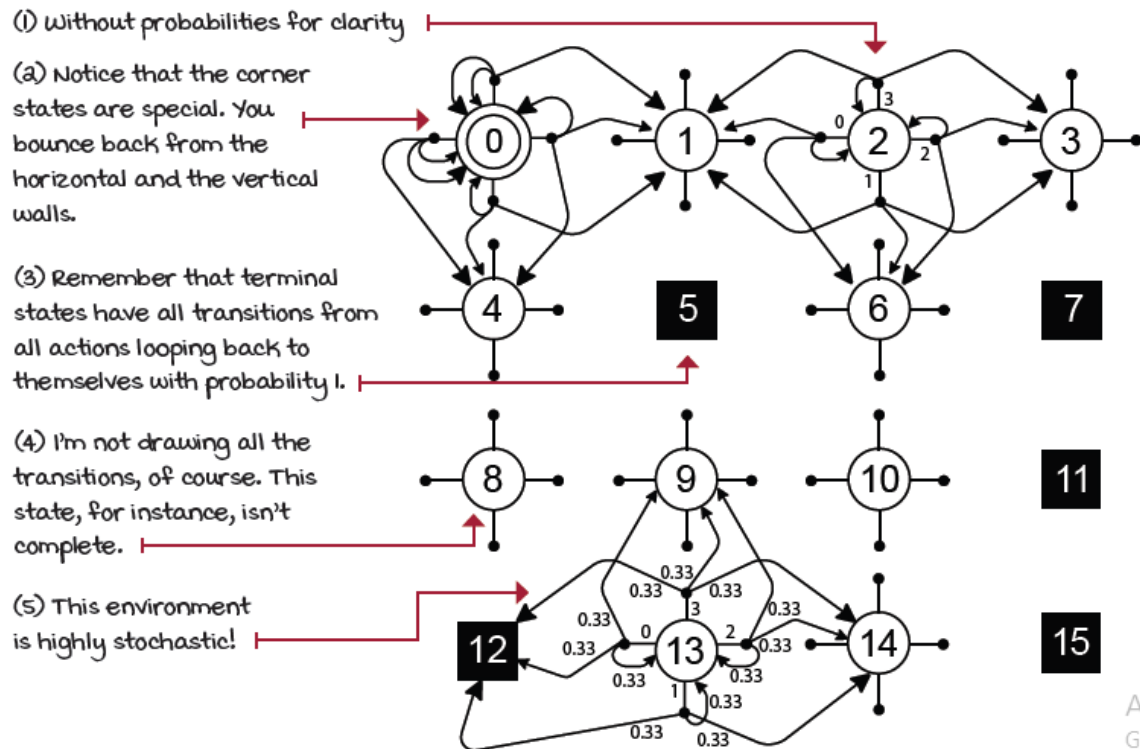
$$p(s'|s, a) = P(S_t = s' | S_{t-1} = s, A_{t-1} = a)$$

(4) Given these are probabilities, we expect the sum of the probabilities across all possible next states to sum to 1.

$$\sum_{s' \in S} p(s'|s, a) = 1, \forall s \in S, \forall a \in A(s)$$

(5) That's true for all states s in the set of states S , and all actions a in the set of actions available in state s .

The transition function of the frozen lake environment



Reward signal

The reward function R maps a transition tuple s, a, s' to a scalar. The reward function gives a numeric signal of goodness to transitions.

It's also important to highlight that while the reward function can be represented as $R(s,a,s')$, which is explicit, we could also use $R(s,a)$, or even $R(s)$, depending on our needs. Sometimes rewarding the agent based on state is what we need; sometimes it makes more sense to use the action and the state. However, the most explicit way to represent the reward function is to use a state, action, and next state triplet. With that, we can compute the marginalization over next states in $R(s,a,s')$ to obtain $R(s,a)$, and the marginalization over actions in $R(s,a)$ to get $R(s)$. But, once we're in $R(s)$ we can't recover $R(s,a)$ or $R(s,a,s')$, and once we're in $R(s,a)$ we can't recover $R(s,a,s')$.

(1) The reward function can be defined as follows.

(2) It can be defined as a function that takes in a state-action pair.

$$r(s, a) = \mathbb{E}[R_t | S_{t-1} = s, A_{t-1} = a]$$

(3) And, it's the expectation of reward at time step t , given the state-action pair in the previous time step.

(4) But, it can also be defined as a function that takes a full transition tuple s, a, s' .

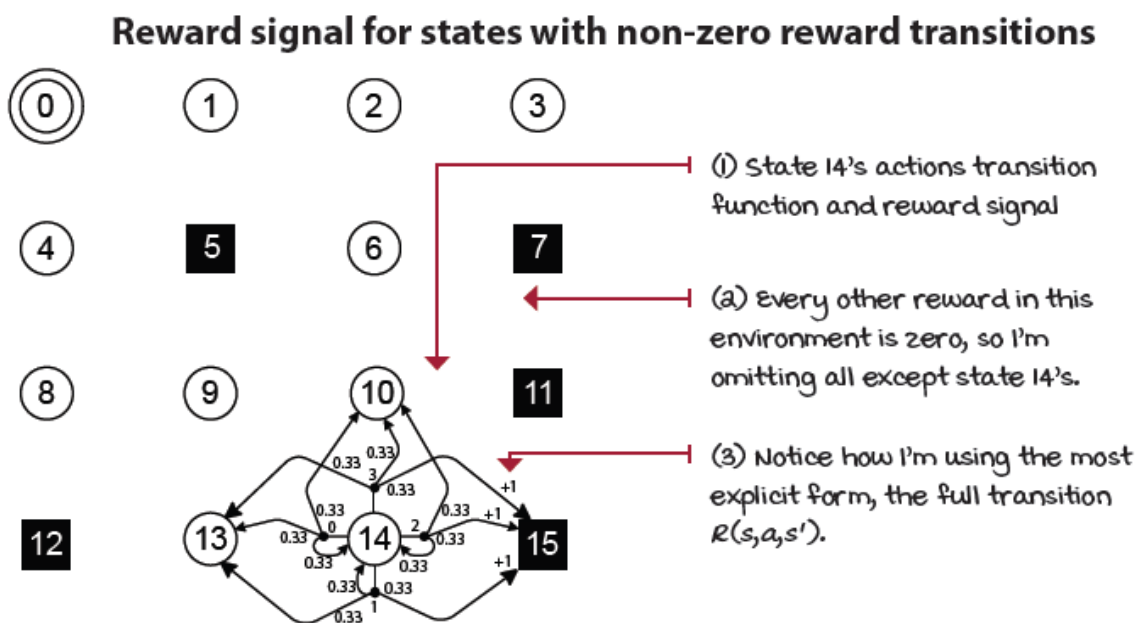
(5) And it's also defined as the expectation, but now given that transition tuple.

$$r(s, a, s') = \mathbb{E}[R_t | S_{t-1} = s, A_{t-1} = a, S_t = s']$$

(6) The reward at time step t comes from a set of all rewards $\mathcal{R}$, which is a subset of all real numbers.

$$R_t \in \mathcal{R} \subset \mathbb{R}$$

There are only three ways to land on 15. **(1)** Selecting the Right action in state 14 will transition the agent with 33.3% chance there (33.3% to state 10 and 33.3% back to 14). But, **(2)** selecting the Up and **(3)** the Down action from state 14 will unintentionally also transition the agent there with 33.3% probability for each action.



Expanding the transition and reward functions into a table form is also useful. The following is the format I recommend for most problems. Notice that I've only added a subset of the

transitions (rows) to the table to illustrate the exercise. Also notice that I'm being explicit, and several of these transitions could be grouped and refactored (for example, corner cells).

State	Action	Next state	Transition probability	Reward signal
0	Left	0	0.33	0
0	Left	0	0.33	0
0	Left	4	0.33	0
0	Down	0	0.33	0
0	Down	4	0.33	0
0	Down	1	0.33	0
0	Right	4	0.33	0
0	Right	1	0.33	0
0	Right	0	0.33	0

State	Action	Next state	Transition probability	Reward signal
0	Up	1	0.33	0
0	Up	0	0.33	0
0	Up	0	0.33	0
1	Left	1	0.33	0
1	Left	0	0.33	0
1	Left	5	0.33	0
1	Down	0	0.33	0
1	Down	5	0.33	0
1	Down	2	0.33	0
1	Right	5	0.33	0
1	Right	2	0.33	0
1	Right	1	0.33	0

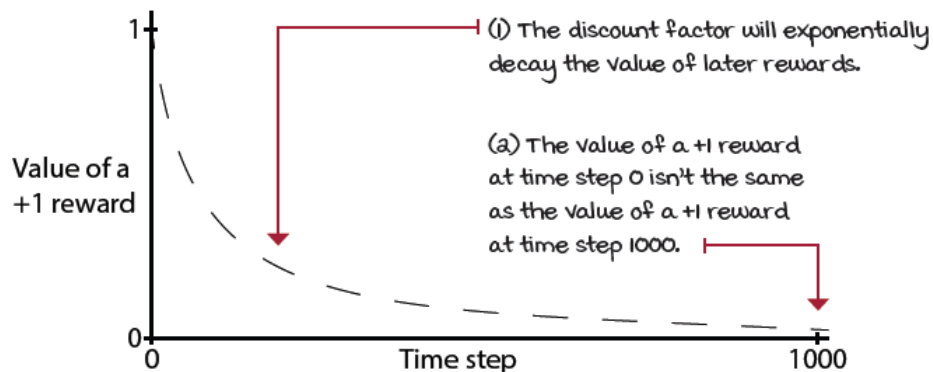
2	Left	1	0.33	0
2	Left	2	0.33	0
2	Left	6	0.33	0
2	Down	1	0.33	0
...	...	...	...	...
14	Down	14	0.33	0
14	Down	15	0.33	1
14	Right	14	0.33	0
14	Right	15	0.33	1
14	Right	10	0.33	0
14	Up	15	0.33	1
14	Up	10	0.33	0
...	...	...	...	...
15	Left	15	1.0	0
15	Down	15	1.0	0
15	Right	15	1.0	0
15	Up	15	1.0	0

Discount Factor

Because of the possibility of infinite sequences of time steps in infinite horizon tasks, we need a way to discount the value of rewards over time; that is, we need a way to tell the agent that getting +1's is better sooner than later. We commonly use a positive real value less than one to exponentially discount the value of future rewards. The further into the future we receive the reward, the less valuable it is in the present.

This number is called the **discount factor**, or **gamma**. The discount factor adjusts the importance of rewards over time. The later we receive rewards, the less attractive they are to present calculations. Another important reason why the discount factor is commonly used is to reduce the variance of return estimates. Given that the future is uncertain, and that the further we look into the future, the more stochasticity we accumulate and the more variance our value estimates will have, the discount factor helps reduce the degree to which future rewards affect our value function estimates, which stabilizes learning for most agents.

Effect of discount factor and time on the value of rewards



Interestingly, gamma is part of the MDP definition: the problem, and not the agent. However, often you'll find no guidance for the proper value of gamma to use for a given environment. Again, this is because gamma is also used as a hyperparameter for reducing variance, and therefore left for the agent to tune. For the BW and BSW environments, a gamma of 1 is appropriate; for the FL environment, however, we'll use a gamma of 0.99, a commonly used value.

(i) The sum of all rewards obtained during the course of an episode is referred to as the return.

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

(a) But we can also use the discount factor this way and obtain the discounted return. The discounted return will downweight rewards that occur later during the episode.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T$$

(3) We can simplify the equation and have a more general equation, such as this one.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

(4) Finally, take a look at this interesting recursive definition. In the next chapter, we spend time exploiting this form.

$$G_t = R_{t+1} + \gamma G_{t+1}$$

$$\mathcal{MDP}(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{S}_\theta, \gamma, \mathcal{H})$$

(i) MDPs have state space $\mathcal{S}$, action space $\mathcal{A}$, transition function $\mathcal{T}$, reward signal $\mathcal{R}$. They also have a set of initial states distribution $\mathcal{S}_\theta$, the discount factor γ , and the horizon $\mathcal{H}$.

$$\mathcal{POMDP}(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{S}_\theta, \gamma, \mathcal{H}, \mathcal{O}, \mathcal{E})$$

(a) To define a POMDP, you add the observation space $\mathcal{O}$ and an emission probability $\mathcal{E}$ that defines the probability of showing an observation o_t given a state s_t . Very simple.